

Smallest Subsequence of Distinct Characters | Greedy | Monotonic Stack |

```
class Solution {  
    public String smallestSubsequence(String s) {  
        int[] freq = new int[27];  
        boolean[] seen = new boolean[27];  
        Stack<Character> stack = new Stack<>();  
  
        for (int i = 0; i < s.length(); i++)  
            freq[s.charAt(i) & 31]++;  
  
        for (int i = 0; i < s.length(); i++) {  
            char c = s.charAt(i);  
            int x = c & 31;  
            freq[x]--;  
  
            if (seen[x])  
                continue;  
  
            while (!stack.isEmpty()) {  
                if (stack.peek() <= c)  
                    break;  
  
                if (freq[stack.peek() & 31] == 0)  
                    break;  
  
                seen[stack.peek() & 31] = false;  
                stack.pop();  
            }  
            stack.push(c);  
            seen[x] = true;  
        }  
        return stack.toString();  
    }  
}
```

```

    }

    stack.push(c);
    seen[x] = true;
}

StringBuilder res = new StringBuilder();

for (char c : stack)
    res.append(c);

return res.toString();
}
}

```

Intuition

To construct the **lexicographically smallest subsequence**, we prefer smaller characters to appear as early as possible.

A **monotonic stack** provides an efficient way to discard previously selected characters whenever a better choice becomes available.

Consider the string:

S = "bcaacdcbc"

We test the every characters as we build the stack.

For each character we encounter, there are only a few possible cases:

Case 1

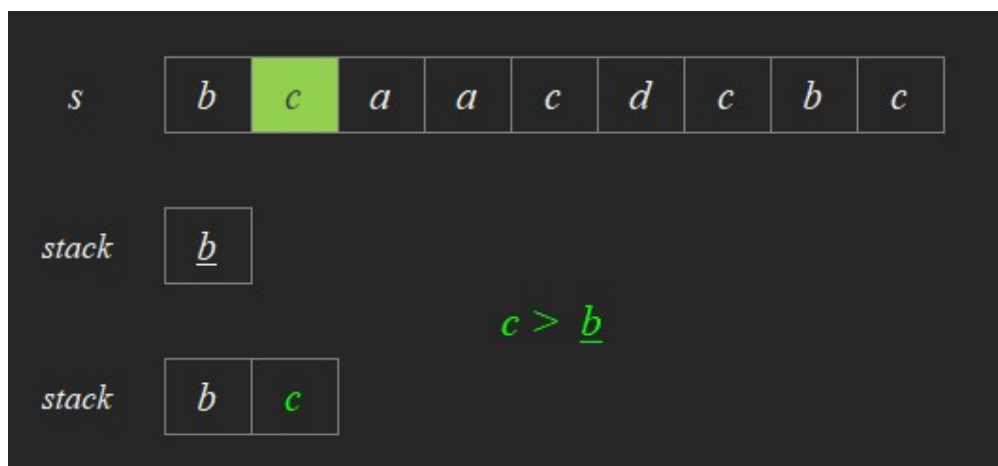
The stack is empty:



There is nothing to compare against, so we simply push the current character.

Case 2

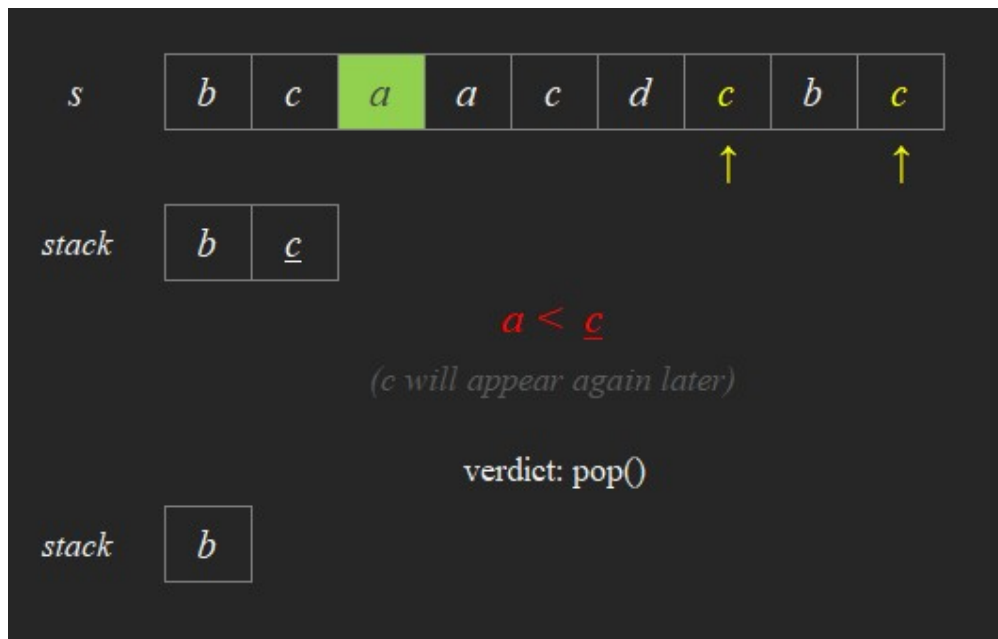
The current character is lexicographically larger than the top element of the stack.



Keeping it keeps the lexicographical order unchanged, so we simply push it onto the stack.

Case 3

The current character is lexicographically smaller than the top element of the stack.

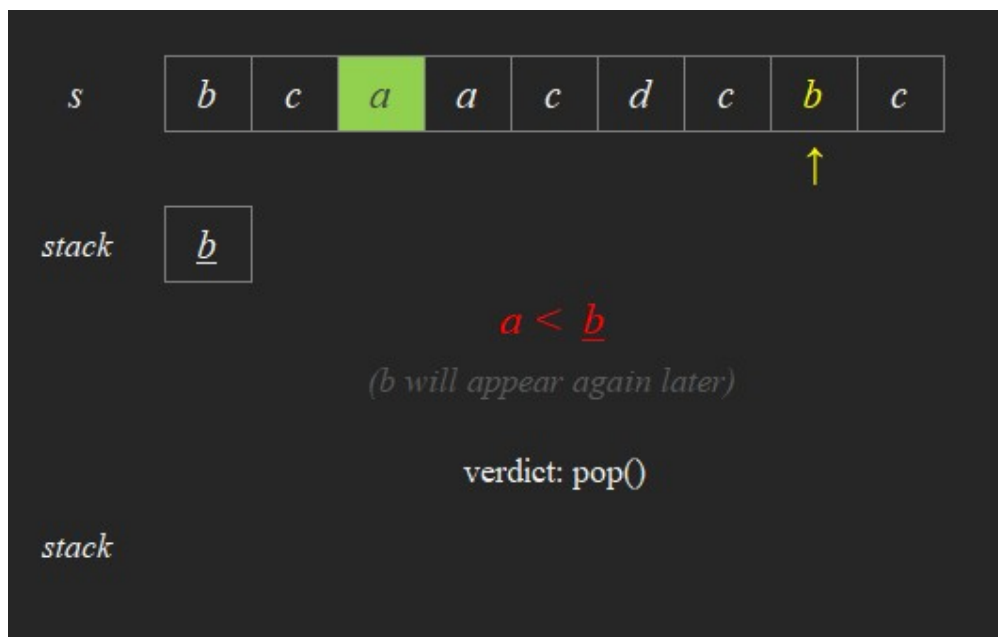


Greedy Selection

Since we want build the **smallest subsequence**, we have to **greedily** ensure that the smallest character appear as early as possible.

Thus, if the larger character (**top element**) will **appear** later in the string:

- We can safely **pop** this without losing this character from the **final subsequence**.

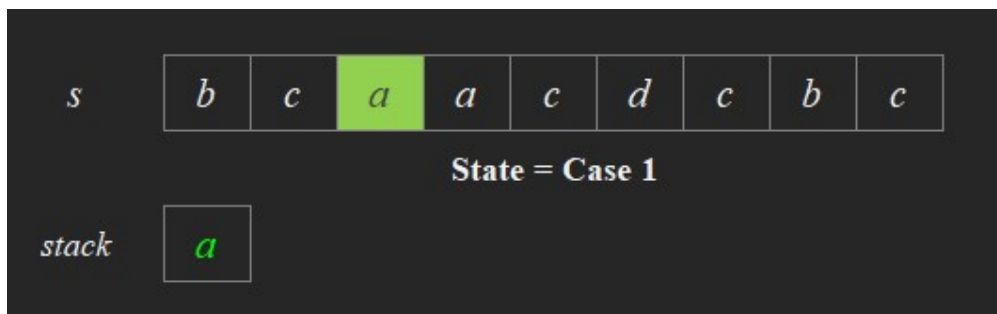


- Then, continue comparing with the new top.

We must repeat this until either:

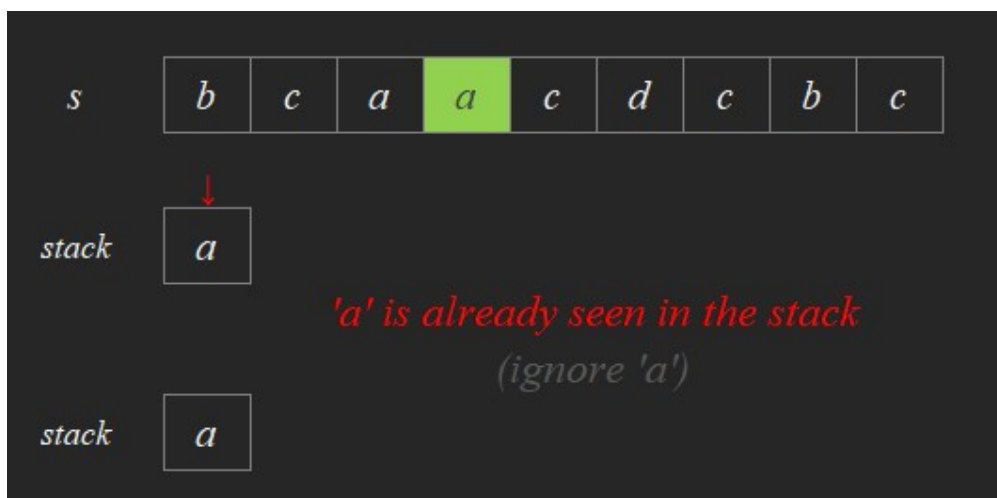
- the stack becomes empty (case 1)
- the top element is lexicographically smaller (case 2),
- or the top element does not appear again later.

Finally, we push the current character.

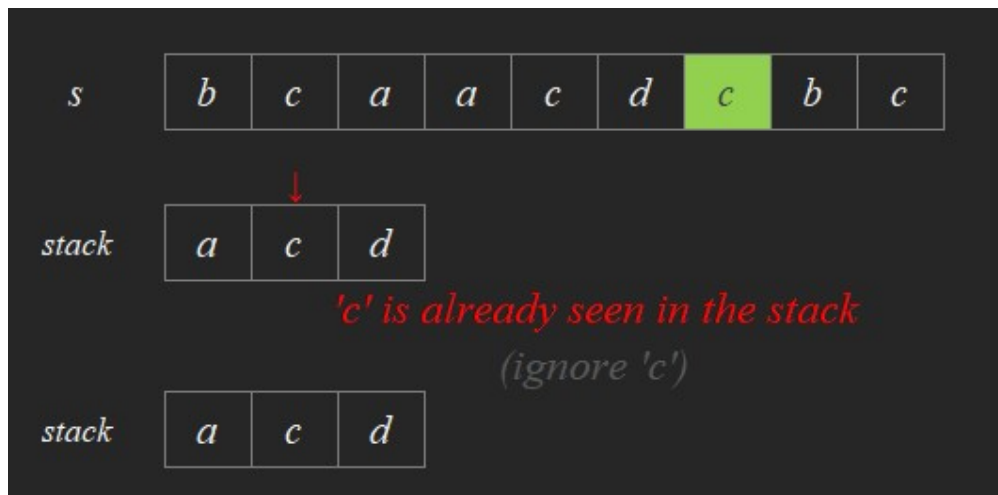


Case 4

The current char is already seen in the stack.



...



Since every distinct character must appear exactly once:

- We conceptually “ignore” this character.

The final state of the stack becomes the resulting final subsequence.



Time Complexity: $O(n)$

Space Complexity: $O(1)$